

Estrutura de Dados

Listas Duplamente Encadeadas

[Baixe aqui os exemplos da aula](#)



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

O Segundo Ponteiro: Anterior

Na lista encadeada simples, cada nó apontava só para a frente, para o próximo da cadeia. Hoje cada nó ganha um segundo ponteiro, anterior, apontando para o nó de trás. A cadeia deixa de ter um único sentido de percurso: a partir de qualquer nó dá para caminhar tanto para a frente quanto para trás, sem precisar recomeçar do início.



Prof. Everaldo Graciliano Souza Ribeiro

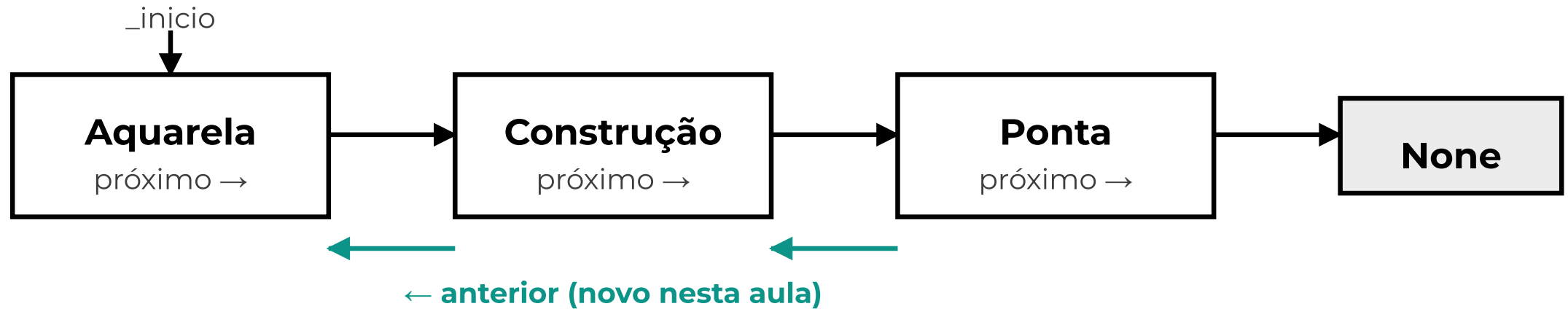
Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Cada Nó Aponta para os Dois Vizinhos



Seguir em frente ou voltar custam o mesmo agora: **O(1)** nos dois sentidos.



Por Que Só "Próximo" Não Bastava

Uma playlist em reprodução precisa voltar uma faixa com a mesma facilidade com que avança. Com um único ponteiro para a frente, voltar exigia recomeçar em `_inicio` e caminhar de novo até o nó anterior ao atual — o mesmo custo $O(n)$ de qualquer acesso por posição. O ponteiro anterior elimina esse percurso: o nó de trás já está a um passo de distância.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Inserir Sem "Olhar Para Trás"

Na lista simples, inserir exigia caminhar até o nó anterior à posição, porque não havia outro jeito de alcançá-lo. Na lista dupla, basta chegar ao nó que hoje ocupa a posição e perguntar a ele mesmo, por seguinte, quem vem antes dele. O algoritmo fica mais simples exatamente porque a estrutura já carrega a resposta.



Prof. Everaldo Graciliano Souza Ribeiro

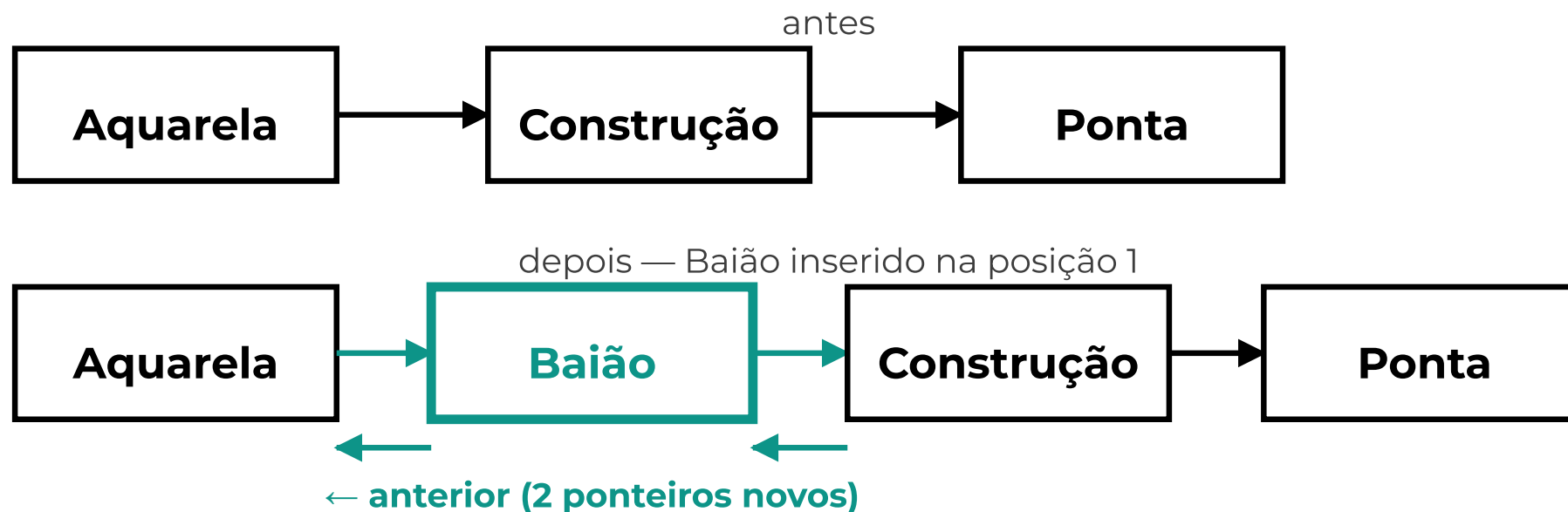
Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Inserção Religando os Dois Sentidos



Inserir religa **quatro** ponteiros agora: dois para frente e dois para trás.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Remover Religando os Dois Lados

Na lista simples, remover religava um único ponteiro: o do nó anterior, que passava a apontar para o nó seguinte ao removido. Na lista dupla, os dois vizinhos do nó removido precisam ser religados entre si — o da esquerda aponta para a frente, o da direita aponta para trás — para que a cadeia continue consistente em ambos os sentidos depois da remoção.



Prof. Everaldo Graciliano Souza Ribeiro

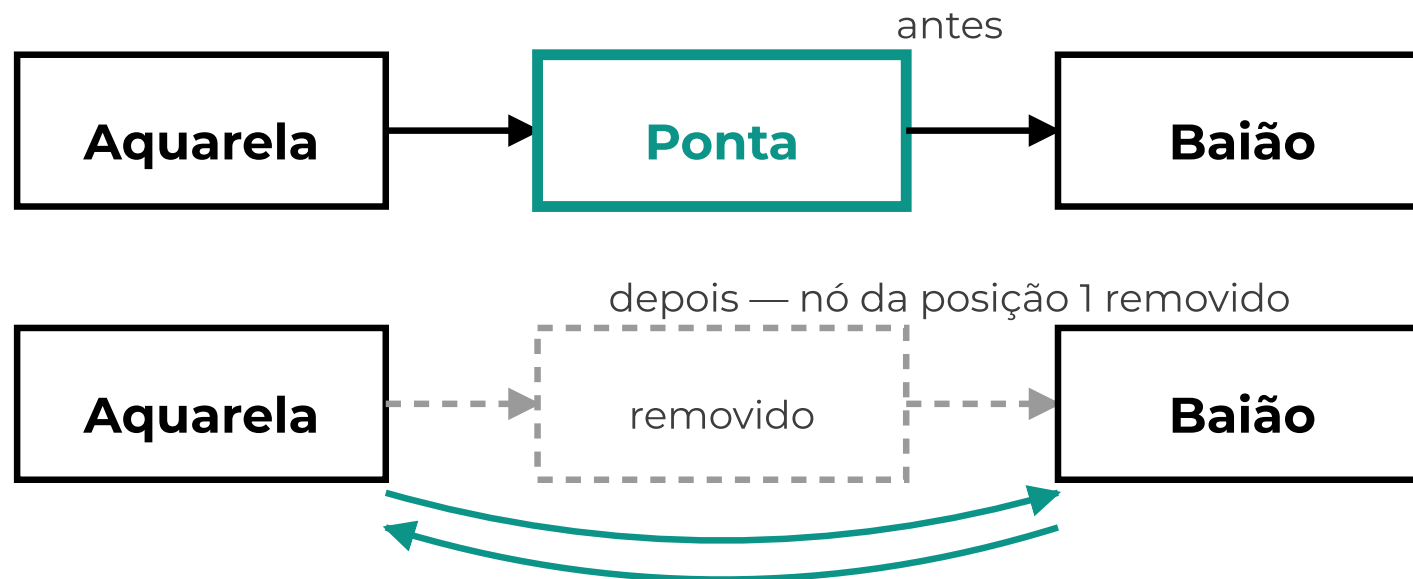
Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Remoção Religando os Dois Lados



Aquarela.próximo e Baião.anterior pulam direto um para o outro — os dois sentidos ficam consistentes.



Navegação de Reprodução em Dois Sentidos

A playlist ganha um cursor: o nó da faixa em reprodução no momento. Avançar segue o ponteiro proximo do cursor, exatamente como já acontecia na Aula 04. Voltar agora segue o ponteiro anterior do cursor — uma operação que, na lista simples, não existia a custo $O(1)$: só seria possível recomeçando em `_inicio` e percorrendo de novo toda a cadeia até ali.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

O Nó com os Dois Ponteiros

```
class _No:
    """Um elo da cadeia: guarda uma faixa e aponta para
    os dois vizinhos."""

    def __init__(self, faixa):
        self.faixa = faixa
        self.proximo = None
        self.anterior = None
```



Inserir: seguinte.anterior Substitui o Percurso

```
def inserir_em(self, posicao, faixa):
    seguinte = self._no_em(posicao) if posicao < self._tamanho else None
    anterior = seguinte.anterior if seguinte is not None else self._fim
    novo = _No(faixa)
    novo.proximo = seguinte
    novo.anterior = anterior
    if anterior is None:
        self._inicio = novo
    else:
        anterior.proximo = novo
    if seguinte is None:
        self._fim = novo
    else:
        seguinte.anterior = novo
    self._tamanho += 1
```



Remover: os Dois Vizinhos se Religam

```
def remover_em(self, posicao):  
    no = self._no_em(posicao)  
    if no.anterior is None:  
        self._inicio = no.proximo  
    else:  
        no.anterior.proximo = no.proximo  
    if no.proximo is None:  
        self._fim = no.anterior  
    else:  
        no.proximo.anterior = no.anterior  
    self._tamanho -= 1  
    return no.faixa
```



Reprodução: próxima e anterior

```
def proxima(self):  
    if self._cursor is None or self._cursor.proximo is None:  
        raise IndexError("nao ha proxima faixa")  
    self._cursor = self._cursor.proximo  
    return self._cursor.faixa  
  
def anterior(self):  
    # O(1): o ponteiro anterior evita recommear em _inicio.  
    if self._cursor is None or self._cursor.anterior is None:  
        raise IndexError("nao ha faixa anterior")  
    self._cursor = self._cursor.anterior  
    return self._cursor.faixa
```



Atividades

Exercício 1

Por que `inserir_em` não precisa mais caminhar até o nó anterior à posição, ao contrário da versão da Aula 04?

Exercício 2

Quantos ponteiros são religados ao remover um nó do meio da lista dupla? E na lista simples da Aula 04?

Exercício 3

Por que `anterior()` custa $O(1)$ na lista dupla, mas custaria $O(n)$ se a Playlist ainda fosse a lista simples da Aula 04?



Desafio extra

Opcional

Quem quiser ir além do combinado nesta semana:

1. Criar um método embaralhar() que percorre a playlist e troca a ordem das faixas aleatoriamente, religando os ponteiros dos dois lados.
2. Adicionar um método reproduzindo() que devolve a faixa do cursor atual sem mover a reprodução.

Nada disso é cobrado na entrega. O projeto segue completo sem essas adições.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>